

Spline-SVP v1: Verification, Calibration and Algorithmic Development

1 Purpose

The previous report introduced the mathematical definition of spline-SVP v1, its segment cost, the first validity statistic, and an exhaustive reference oracle. The present note focuses on the next steps required to validate the implementation and prepare a more efficient algorithm.

The main topics are:

- (i) verification of fixed-partition fitting;
- (ii) the structure of the optimization problems in the oracle;
- (iii) sensitivity to the validity threshold γ ;
- (iv) Gaussian-null calibration of the validity statistic;
- (v) a functional dynamic-programming formulation.

2 Verification of fixed-partition fitting

For a fixed partition τ , the fitted boundary-state vector is obtained from

$$z^*(\tau) = \arg \min_z \|y - X_\tau z\|_2^2.$$

This is solved numerically by QR-based least squares.

Although the optimization is standard, the implementation should be checked independently of the exhaustive partition search.

2.1 One-segment check

For the partition

$$\tau = (1, n),$$

the fitted spline contains only one affine piece. Therefore its optimal cost must equal the residual sum of squares from ordinary linear regression:

$$C^*((1, n)) = \min_{a, b} \sum_{i=1}^n (y_i - a - bx_i)^2.$$

This provides a direct numerical comparison between `fit_fixed_partition()` and a standard affine least-squares fit.

2.2 Continuity check

For an interior boundary τ_k , continuity requires

$$\ell_{\tau_{k-1}, \tau_k}^{z_{k-1}, z_k}(x_{\tau_k}) = \ell_{\tau_k, \tau_{k+1}}^{z_k, z_{k+1}}(x_{\tau_k}) = z_k.$$

Thus, after fitting a partition with several segments, the left and right fitted values at every interior boundary can be compared numerically. Their difference should be zero up to floating-point precision.

2.3 Independent optimization check

For a small partition, the least-squares solution may also be compared with a generic numerical minimizer applied directly to

$$C(\tau, z).$$

The two methods should give the same minimum cost up to numerical tolerance.

This comparison is useful as an independent verification of the design matrix construction, although least squares remains the preferred implementation.

3 Finite and continuous optimization

The current reference algorithm contains two different types of optimization.

First, the partition itself belongs to a finite set. Since each interior index

$$2, \dots, n-1$$

is either selected or not selected as a boundary, the number of candidate partitions is

$$2^{n-2}.$$

The exhaustive oracle explicitly enumerates these possibilities.

Second, for a fixed partition, the boundary-state vector satisfies

$$z \in \mathbb{R}^{K+1}.$$

The values of z are therefore not enumerated. Instead,

$$\min_z C(\tau, z)$$

is solved as a continuous least-squares problem.

The same distinction occurs inside the validity statistic. For a fixed candidate knot r ,

$$RSS_1(s, r, t) = \min_{a, b, c} \sum_{i=s+1}^t [y_i - a - bx_i - c(x_i - x_r)_+]^2$$

is a continuous least-squares problem. However, the knot position is chosen from the finite set

$$r \in \{s+1, \dots, t-1\}.$$

Hence the implementation enumerates r but solves (a, b, c) by least squares.

The exponential complexity of the reference oracle therefore comes from enumeration of complete partitions, not from the optimization of the continuous parameters.

4 Effect of the threshold γ

The first synthetic example used

$$\gamma = 4$$

only as a provisional value. A natural next experiment is to keep the same dataset fixed and solve the problem for several thresholds, for example

$$\gamma \in \{0.5, 1, 2, 4, 8, 16, 32\}.$$

For each threshold, the quantities

$$\gamma, \quad K^*, \quad \hat{\tau}, \quad C^*(\hat{\tau})$$

should be recorded.

Increasing γ makes the validity condition

$$T_{s,t} \leq \gamma$$

less restrictive. This gives the following monotonicity property.

Proposition 1. *If*

$$\gamma_1 \leq \gamma_2,$$

then

$$K^*(\gamma_2) \leq K^*(\gamma_1).$$

Indeed, every segment valid under γ_1 is also valid under γ_2 . Therefore

$$\mathcal{V}_{\gamma_1} \subseteq \mathcal{V}_{\gamma_2},$$

so enlarging the threshold cannot increase the minimum feasible number of segments.

This gives a useful numerical check of the implementation: $K^*(\gamma)$ should be non-increasing as γ increases.

The secondary cost

$$C^*(\hat{\tau})$$

does not necessarily have the same monotonicity because the number of segments is optimized first. When K^* changes, the estimator moves to a different class of candidate partitions.

5 Gaussian-null calibration

The threshold γ should ultimately be chosen from the distribution of the validity statistic under a true affine segment.

For a candidate interval, simulate

$$y_i = a + bx_i + \varepsilon_i, \quad \varepsilon_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2),$$

and compute

$$T_{s,t} = \frac{1}{\sigma^2} \left[RSS_0(s, t) - \min_{s < r < t} RSS_1(s, r, t) \right].$$

Repeating this simulation B times gives

$$T^{(1)}, \dots, T^{(B)},$$

from which an empirical threshold can be obtained. For a target upper-tail probability α ,

$$\hat{\gamma}_{m,\alpha} = \hat{Q}_{1-\alpha} \left(T^{(1)}, \dots, T^{(B)} \right),$$

where

$$m = t - s$$

is the number of observations in the interval.

For example, with

$$\alpha = 0.05,$$

the calibrated threshold approximately satisfies

$$\Pr_{\text{null}} (T_{s,t} > \hat{\gamma}_{m,0.05}) \approx 0.05.$$

This experiment can also verify expected invariance properties. Since the affine intercept and slope are estimated under the null, changing their values should not materially affect the null distribution. Similarly, multiplying both the observations and σ by the same factor should leave $T_{s,t}$ unchanged.

5.1 Dependence on interval length

A key question is whether one threshold γ can be used for all segment lengths.

The statistic contains a maximization over candidate knot locations:

$$T_{s,t} = \max_{s < r < t} \frac{RSS_0(s, t) - RSS_1(s, r, t)}{\sigma^2}.$$

As the interval length increases, the number of candidate values of r also increases. Consequently, the null distribution of the maximum may depend on

$$m = t - s.$$

A first experiment can compare lengths such as

$$m \in \{5, 10, 20, 50, 100\}$$

and estimate, for example, the empirical 90%, 95%, and 99% quantiles of $T_{s,t}$.

If the quantiles vary substantially with m , then a length-dependent threshold

$$\gamma = \gamma(m, \alpha)$$

may be more appropriate than one universal constant.

6 Dynamic-programming direction

The exhaustive oracle is useful for small- n verification but cannot scale because it evaluates

$$2^{n-2}$$

partitions.

State-independent validity simplifies the transition to dynamic programming because the quantities

$$V_{s,t} = \mathbf{1}\{T_{s,t} \leq \gamma\}$$

can be computed before solving for any boundary states.

If only the number of valid segments were minimized, one could use the scalar recurrence

$$F(t) = 1 + \min_{\substack{s < t \\ V_{s,t}=1}} F(s).$$

The continuous fitting cost requires additional state information, because the cost of a segment depends on its endpoint values. Define

$$Q_t(v)$$

as the best objective up to boundary t , conditional on the fitted value at x_t being v . A natural functional recurrence is

$$Q_t(v) = \operatorname{lexmin}_{\substack{s < t \\ V_{s,t}=1}} \inf_{u \in \mathbb{R}} \{Q_s(u) + (1, C_{s,t}(u, v))\}.$$

The first component counts segments and the second component contains the residual fitting cost, so lexicographic comparison preserves the definition of spline-SVP.

Since

$$C_{s,t}(u, v)$$

is quadratic in (u, v) , this recurrence suggests a piecewise-quadratic representation of the value functions Q_t . The main unresolved question is whether the number of candidate quadratic pieces can be controlled efficiently by exact pruning.

The exhaustive oracle will provide the reference solution against which such a functional implementation can be checked.

7 Next steps

The immediate objectives are:

- (i) complete direct numerical tests of `fit_fixed_partition()`;
- (ii) run the fixed synthetic example for several values of γ ;
- (iii) check the monotonic behavior of $K^*(\gamma)$;
- (iv) perform Gaussian-null calibration for several interval lengths;
- (v) compare the resulting empirical thresholds;
- (vi) begin implementation of the functional recurrence and compare its solutions with the exhaustive oracle.

These steps separate numerical correctness, statistical calibration, and algorithmic efficiency, while keeping the exhaustive solver as a reference throughout the development.